DASE 7506 · SEMESTER 1 · SESSION 4

Advanced Machine Learning Methods

Fine-tuning — from a spam classifier to an instruction-following assistant, and the adapters that make it cheap.

Jiayu Chen · 23 September 2026 · Raschka, chapters 6 and 7, appendix E

SESSION 4

Today’s Three Blocks

01

Fine-Tuning for Classification

Chapter 6. When the fine-tuning taxonomy splits, and the small last change that turns a text generator into a grammar.

50 min

02

Fine-Tuning to Follow Instructions

Chapter 7. Prompt templates, masked targets, and the problem of evaluating a model whose output is free text.

50 min

03

Beyond Full Fine-Tuning

Appendix E. Low-rank adapters, the variants that replaced the original recipe, and the sparse-expert models.

50 min

PART I · CHAPTER 6

Fine-Tuning for Classification

The smallest change to a pretrained GPT that turns it into a classifier.

CHAPTER 6 · OVERVIEW

Where Chapter 6 Sits in the Build

The three-stage picture from Session 1, with everything you have already written struck through.

Stage 1

Sessions 1–2 · done

1 Data preparation and sampling

2 Attention mechanism

3 LLM architecture

Stage 2

Session 3 · done

5 Pretraining

6 Model evaluation

7 Load pretrained weights

Stage 3

Today

8 Fine-tuning → classifier

9 Fine-tuning → personal assistant

CHAPTER 6.1

Two Categories of Fine-Tuning

CLASSIFICATION FINE-TUNING

The model is trained to recognise a fixed set of class labels. The input carries no instruction. The output space is the label set and nothing else.

```
"You are a winner you have been
specially selected to receive
$1000 cash."

  -> spam
```

It cannot tell you *why*, or summarise, or answer anything else. Specialised models are easier to build than generalists.

INSTRUCTION FINE-TUNING

The model is trained on tasks described in natural language. The instruction is part of the input. The output is free text.

```
Is the following text 'spam'?
"You are a winner ..."
Answer with 'yes' or 'no'.

  -> Yes.
```

The same model translates, summarises and answers questions — and is correspondingly hungrier for data.

CHAPTER 6.1

Choosing Between Them

	Classification fine-tuning	**Instruction fine-tuning**
Output space	A fixed label set	Any token sequence
Input at inference	Raw text, no prompt needed	Instruction, plus optional input
Data required	Hundreds to thousands of labelled pairs	Thousands of instruction–response pairs
Base model size	124 M is enough	355 M is the minimum that works
Compute, this book	5.65 minutes on a laptop	15.78 minutes on a laptop
Evaluation	Accuracy — one number, no argument	Contested; needs a judge
Fails by	Refusing everything outside the label set	Fluent, confident nonsense

CHAPTER 6.2

The Dataset

The SMS Spam Collection from the UCI repository — real text messages, hand-labelled.

5,572
messages in the raw file

747
labelled spam — 13.4% of the total

4,825
labelled ham

1,494
after undersampling to a balanced set

Why undersample rather than weight the loss? With a 6.5:1 imbalance, a model that always answers “ham” scores 86.6% accuracy. Balancing the classes makes accuracy an honest metric and keeps the training loop identical to chapter 5. It also throws away 4,078 messages — a real cost, and the reason production spam filters weight the loss instead.

CHAPTER 6.2 · LIVE CODE

Balancing and Splitting

```
def create_balanced_dataset(df):
    num_spam = df[df["Label"] == "spam"].shape[0]
    ham_subset = df[df["Label"] == "ham"].sample(
        num_spam, random_state=123
    )
    return pd.concat([ham_subset, df[df["Label"]
== "spam"]])

balanced_df = create_balanced_dataset(df)
balanced_df["Label"] = balanced_df["Label"].map(
    {"ham": 0, "spam": 1}
)

train_df, validation_df, test_df = random_split(
    balanced_df, 0.7, 0.1
)
```

70 / 10 / 20

1,045 training · 149 validation · 300 test. The test fraction is the remainder, not a third argument.

Labels become integers

“ham” → 0, “spam” → 1. Cross-entropy wants class indices, not strings.

random_state=123

Fixed, so your numbers match the book’s and each other’s. Report the seed in your mini-project.

CHAPTER 6.3 · LIVE CODE

One Batch, One Length

```
class SpamDataset(Dataset):
    def __init__(self, csv_file, tokenizer,
                 max_length=None, pad_token_id=50256):
        self.data = pd.read_csv(csv_file)
        self.encoded_texts = [tokenizer.encode(t)
                              for t in self.data["Text"]]
        if max_length is None:
            self.max_length = self._longest_encoded_length()
        else: # truncate
            self.max_length = max_length
            self.encoded_texts = [
                t[:max_length] for t in self.encoded_texts
            ]
        self.encoded_texts = [ # then pad
            t + [pad_token_id] * (self.max_length - len(t))
            for t in self.encoded_texts
        ]
```

120

tokens — the longest message in the training set, and therefore the length of every message

Validation and test inherit it

They are constructed with **max_length=train_dataset.max_length**. A longer test message is truncated; a shorter one is padded with 50256.

CHAPTER 6.3

The Loaders

130

training batches — 1,045 examples at batch size 8, with drop_last=True

19

validation batches

38

test batches

Input batch

torch.Size([8, 120]) — eight messages, 120 token IDs each.

Label batch

torch.Size([8]) — one integer per message. No sequence dimension: the label describes the whole message.

drop_last=True

The last partial training batch is discarded so every batch has the same shape. Validation and test keep theirs.

CHAPTER 6.4

What the Base Model Does With the Task

IT GENERATES FINE

```
text = "Every effort moves you"

-> "Every effort moves you forward.
    The first step is to understand
    that..."
```

The 124 M weights loaded in Session 3 are intact and the model is a competent text continuer.

IT CANNOT CLASSIFY

```
text = ("Is the following text 'spam'? "
        "Answer with 'yes' or 'no': "
        "You are a winner...")

-> "...'spam'? Answer with 'yes' or
    'no': You are a winner..."
```

It continues the prompt instead of answering it. The base model has never been shown that an instruction is a request.

CHAPTER 6.5

Replacing the Output Head

BEFORE

```
out_head: Linear(768, 50257)
```

One score per vocabulary entry. 38,597,376 parameters — 31% of the whole model.



AFTER

```
out_head: Linear(768, 2)
```

One score per class. 1,538 parameters. Randomly initialised, and the only layer that must be trained.

Everything below the head — embeddings, twelve transformer blocks, the final norm — is unchanged. The features that let the model predict the next word are the same features that let it tell spam from ham.

CHAPTER 6.5 · LIVE CODE

Freeze, Replace, Unfreeze

```
# 1. freeze everything
for param in model.parameters():
    param.requires_grad = False

# 2. swap the head; new layers are trainable by default
torch.manual_seed(123)
model.out_head = torch.nn.Linear(
    in_features=768, out_features=2
)

# 3. thaw the top of the stack
for param in model.trf_blocks[-1].parameters():
    param.requires_grad = True
for param in model.final_norm.parameters():
    param.requires_grad = True
```

Trainable after step 1

0 of 124,441,346

Trainable after step 3

7,090,946 — the last transformer block, the final LayerNorm and the new head. 5.7% of the model.

Why the last block

The most task-specific representations in the network. Lower blocks encode syntax this task need not relearn.

How Much Should You Unfreeze?

Configuration	Trainable parameters	Test accuracy
Output head only	1,538	Substantially worse — the head alone cannot adapt the features
Last block + final norm + head (the chapter)	7,090,946	95.67%
The entire model (exercise 6.2)	124,441,346	96.67%

One point of accuracy

for seventeen times the trainable parameters. Whether that is a good trade is a deployment question, not a modelling one.

The third block returns to this

LoRA reaches **98.00%** on the same split with 2.7 M trainable parameters. Full fine-tuning is not the ceiling it looks like here.

CHAPTER 6.5

Which Token Carries the Prediction

The model produces a 2-vector at *every* one of the 120 positions. We read only the last one.

```
logits = model(input_batch) # [8, 120, 2]
logits = logits[:, -1, :] # [8, 2]
```

Because attention is causal, position *t* has attended to positions 1 through *t* and no further. Only the final position has seen the whole message.

An encoder would differ

BERT attends in both directions, so any position — a prepended [CLS] token, by convention — can carry the summary. A decoder has one such position, and it is the last.

The padding question

For every message shorter than 120, that last position *is* a padding token. It works because attention has already aggregated the real tokens into it.

CHAPTER 6.6

From Logits to a Label

RAW OUTPUT

01 · Logits

[-3.5983, 3.9902]

Two raw scores from the new head at the final position.

NORMALISED

02 · Probabilities

softmax → [0.0006, 0.9994]

Needed to report confidence, not to pick the answer.

DECISION

03 · Label

argmax → 1 → “spam”

Softmax is monotonic, so argmax of the logits gives the same class.

46.25%

training accuracy before any fine-tuning

45.00%

validation

48.75%

test

A coin flip, as it must be: the head is randomly initialised and has never seen a label.

Loss and Accuracy on One Position

```
def calc_loss_batch(input_batch, target_batch, model,
device):
    logits = model(input_batch.to(device))[:, -1, :]
    return torch.nn.functional.cross_entropy(
        logits, target_batch.to(device)
    )


# inside calc_accuracy_loader, per batch:
with torch.no_grad():
    logits = model(input_batch.to(device))[:, -1, :]
predicted_labels = torch.argmax(logits, dim=-1)
correct += (predicted_labels == target_batch).sum().item()
```

Why not optimise accuracy directly?

Accuracy is a count of correct argmaxes. It is piecewise constant, so its gradient is zero almost everywhere. Cross-entropy is the differentiable surrogate we can actually descend.

One loss per message

Chapter 5 averaged over every token position. Here there is exactly one supervised position per example, so the batch loss is a mean over 8 numbers, not 8 × 120.

CHAPTER 6.7

The Training Loop, and What Changed

	Chapter 5 · pretraining	Chapter 6 · classification
Supervised positions	Every token in the window	The final token only
Progress counter	tokens_seen	examples_seen
Periodic check	Generate a text sample	Compute accuracy on both splits
Optimiser	AdamW, lr 4e-4, wd 0.1	AdamW, lr 5e-5, wd 0.1
Epochs	10 on 5,145 tokens	5 on 1,045 examples
Starting weights	Random	Pretrained, 124 M

The learning rate is **eight times smaller** than in pretraining. Fine-tuning starts from a good solution; a large step destroys it before the gradient signal from 1,045 examples can rebuild it.

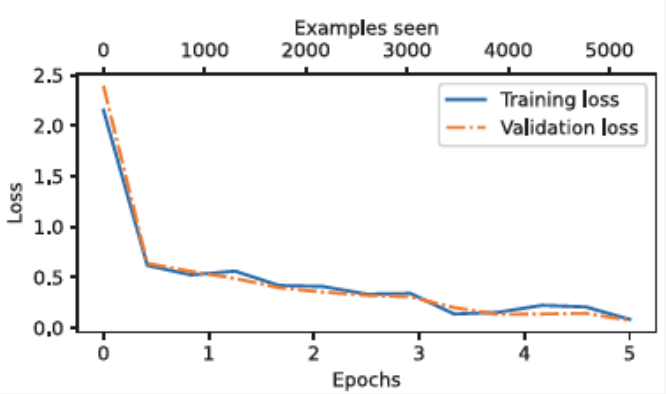
CHAPTER 6.7

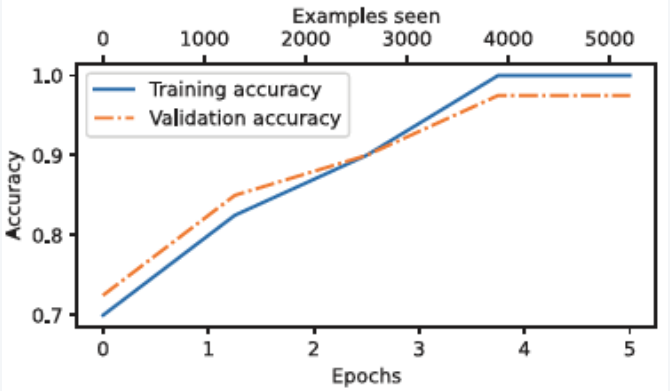
Five Epochs, Five Minutes

Step	Train loss	Val loss	Train acc	Val acc
Ep 1 · 000	2.153	2.392	—	—
Ep 1 · 100	0.523	0.557	70.00%	72.50%
Ep 2 · 250	0.409	0.353	82.50%	85.00%
Ep 3 · 350	0.340	0.306	90.00%	90.00%
Ep 4 · 500	0.222	0.137	100.00%	97.50%
Ep 5 · 600	0.083	0.074	100.00%	97.50%

5.65 minutes

on an M3 MacBook Air. Under half a minute on a V100 or A100.





Using the Classifier

```
def classify_review(text, model, tokenizer, device,
                    max_length=None, pad_token_id=50256):
    model.eval()
    input_ids = tokenizer.encode(text)
    supported = model.pos_emb.weight.shape[0]
    input_ids = input_ids[:min(max_length, supported)]
    input_ids += [pad_token_id] * (max_length - len(input_ids))

    tensor = torch.tensor(input_ids,
device=device).unsqueeze(0)
    with torch.no_grad():
        logits = model(tensor)[:, -1, :]
    predicted_label = torch.argmax(logits, dim=-1).item()
    return "spam" if predicted_label == 1 else "not spam"
```

“You are a winner you have been specially selected to receive $1000 cash…”

→ **spam**

“Hey, just wanted to check if we’re still on for dinner tonight?”

→ **not spam**

RECAP

Block I — Fine-Tuning for Classification

01 Classification fine-tuning fixes the output space to a label set; instruction fine-tuning leaves it as free text.

02 The architectural change is one layer: Linear(768, 50257) becomes Linear(768, 2). Thirty-eight million parameters out, one and a half thousand in.

03 Only the final token’s logits are supervised, because causal attention means only that position has read the whole message.

04 Freezing all but the last block, the final norm and the head costs one point of accuracy against full fine-tuning, and saves seventeen times the trainable parameters.

05 95.67% test accuracy in 5.65 minutes on a laptop — and a 475 MB checkpoint per task, which is the problem the third block exists to solve.

PART II · CHAPTER 7

Fine-Tuning to Follow Instructions

The architecture does not change at all. Every difficulty moves into the data and into the evaluation.

CHAPTER 7 · OVERVIEW

The Three Stages of Chapter 7

Nine steps. Five of them are data handling, and that is the honest proportion.

Stage 1

Preparing the dataset

1. Download and format
2. Batch the dataset
3. Create data loaders

Stage 2

Fine-tuning the LLM

4. Load a pretrained LLM
5. Instruction fine-tune it
6. Inspect the modelling loss

Stage 3

Evaluating the LLM

7. Extract responses
8. Qualitative evaluation
9. Score the responses

CHAPTER 7.1

What Actually Changes

UNCHANGED

Architecture

Nothing. Same GPTModel, same blocks, same output head over the full 50,257-token vocabulary.

UNCHANGED

Objective

Nothing. Cross-entropy on next-token prediction, exactly as in chapter 5.

THE WHOLE CHAPTER

Data

Everything. 1,100 hand-written pairs, wrapped in a fixed template, with the padding masked out of the loss.

Instruction following is not a capability we add to the network. It is a **format** we teach it. The model already knows how to continue text; we are showing it text in which a question is followed by an answer.

CHAPTER 7.2

The Dataset

1,100
instruction–response pairs, written for this book

3
fields per entry

Two shapes of entry

Some entries have an **input** field — a piece of text the instruction operates on. Some have an empty one, where the instruction is self-contained. The template has to handle both.

```
{
  "instruction": "Identify the correct spelling
                  of the following word.",
  "input": "Ocassion",
  "output": "The correct spelling is 'Occasion'."
}

{
  "instruction": "What is an antonym of
                  'complicated'?",
  "input": "",
  "output": "An antonym of 'complicated' is
             'simple'."
}
```

CHAPTER 7.2

Two Prompt Styles

ALPACA STYLE — USED THROUGHOUT THIS CHAPTER

```
Below is an instruction that describes a task.
Write a response that appropriately completes
the request.

### Instruction:
Identify the correct spelling of the following
word.

### Input:
Ocassion

### Response:
The correct spelling is 'Occasion'.
```

PHI-3 STYLE — MICROSOFT

```
<|user|>
Identify the correct spelling of the following
word: 'Ocassion'

<|assistant|>
The correct spelling is 'Occasion'.
```

The template is a contract

Whatever markers you train on must be the markers you prompt with. A model trained on `### Response:` and prompted without it keeps writing instructions.

Building the Prompt

```
def format_input(entry):
    instruction_text = (
        f"Below is an instruction that describes a task. "
        f"Write a response that appropriately completes "
        f"the request."
        f"\n\n### Instruction:\n{entry['instruction']}"
    )
    input_text = (
        f"\n\n### Input:\n{entry['input']}"
        if entry["input"] else ""
    )
    return instruction_text + input_text


model_input = format_input(data[50])
desired_response = f"\n\n### Response:\n{data[50]['output']}"
```

The conditional matters

An empty input field produces no `### Input:` header at all. Emitting an empty section would teach the model that blank sections are normal, and it will start producing them.

Prompt and answer are concatenated

The training example is **format_input(entry) + response_text** — one continuous string. There is no separate “input” and “label” tensor: the label is the string shifted by one.

CHAPTER 7.2

The Split

935

training entries — 85%

55

validation — 5%

110

test — 10%

Why validation is the smallest

It is used only to watch for overfitting during the run. Fifty-five examples give a noisy loss, which is adequate for that one job.

Why test is larger than validation

Every test entry gets scored individually by a judge model at the end of the chapter. 110 scores averaged is about the minimum for a number you would quote.

Compare with chapter 6

1,045 / 149 / 300 at 70 / 10 / 20. Classification could afford a bigger test set because scoring it is free.

CHAPTER 7.3

Batching, in Five Steps

FORMAT
2.1
Apply the prompt template to the entry, and append the response.

TOKENIZE
2.2
Tokenize to IDs. Append one end-of-text token, 50256.

PAD
2.3
Pad every sequence in *this batch* to the longest sequence in it.

SHIFT
2.4
Targets are the inputs shifted one position left.

MASK
2.5
Replace all but one padding token in the targets with −100.

Steps 2.3 and 2.5 are the two that did not appear in chapter 6, and they are the two that cause almost all the bugs. Everything happens inside a **custom collate function** handed to the DataLoader.

CHAPTER 7.3

Pad to the Batch, Not the Dataset

CHAPTER 6 — GLOBAL PADDING

```
[ 40, 1489, 257, ... ] + [50256] * 94 # len 120
[ 71, 521, ... ] + [50256] * 108 # len 120
[ 464, 2068, ... ] + [50256] * 61 # len 120
```

Simple, rectangular, and wasteful. Every batch is 120 wide whether or not anything in it is long.

CHAPTER 7 — PER-BATCH PADDING

```
batch A: max len 78 -> [8, 78]
batch B: max len 214 -> [8, 214]
batch C: max len 46 -> [8, 46]
```

Batches have different widths. Attention is quadratic in width, so the short batches are much cheaper — and most batches are short.

Padding is not free

It is computed, attended over, then discarded. On a long-tailed length distribution, global padding can put most of your FLOPs into token 50256.

The next refinement

Sort by length and batch similar lengths together — *length-grouped batching*. Standard in every serious library.

CHAPTER 7.3

Targets Are the Inputs Shifted

ONE EXAMPLE, PADDED TO 5

```
inputs [ 1, 2, 3, 4, 50256 ]
targets [ 2, 3, 4, 50256, 50256 ]
```

Exactly the supervision signal from chapter 5 — token *t* predicts token *t*+1 — applied to a formatted string instead of a novel.

AFTER MASKING

```
inputs [ 1, 2, 3, 4, 50256 ]
targets [ 2, 3, 4, 50256, -100 ]
```

One end-of-text token survives in the targets. The rest become −100. Everything the model is asked to predict is a real token, plus exactly one stop signal.

Why keep one? If every 50256 were masked, the model would never be supervised on stopping, and at inference it would generate until it hit the token limit. The single retained end-of-text token is the entire mechanism by which the model learns to shut up.

CHAPTER 7.3

Why −100

```
logits = torch.tensor([
    [-1.0, 1.0], # prediction for token 1
    [-0.5, 1.5], # prediction for token 2
    [-0.5, 1.5], # prediction for token 3
])
targets_1 = torch.tensor([0, 1, 1])
loss_1 = F.cross_entropy(logits, targets_1)
# tensor(1.1269)

targets_2 = torch.tensor([0, 1, -100])
loss_2 = F.cross_entropy(logits, targets_2)
# tensor(1.1269) <- the third row contributed nothing
```

ignore_index=-100

the PyTorch default for **cross_entropy**. A target of −100 leaves both the sum and the count, so the mean covers only the surviving positions.

Not the same as padding with 0

Class 0 is a real vocabulary entry; masking with it trains the model to emit that token.

allowed_max_length

The collate function also truncates, so one 900-token example cannot blow up a batch.

CHAPTER 7.4 · LIVE CODE

The Custom Collate Function

```
def custom_collate_fn(batch, pad_token_id=50256,
                      ignore_index=-100, device="cpu"):
    batch_max_length = max(len(item) + 1 for item in batch)
    inputs_lst, targets_lst = [], []
    for item in batch:
        new_item = item + [pad_token_id]
        padded = new_item + [pad_token_id] * (
            batch_max_length - len(new_item))
        inputs = torch.tensor(padded[:-1]) # drop the last
        targets = torch.tensor(padded[1:]) # shift by one
        indices = torch.nonzero(targets == pad_token_id).squeeze()
        if indices.numel() > 1:
            targets[indices[1:]] = ignore_index
        inputs_lst.append(inputs); targets_lst.append(targets)
    return (torch.stack(inputs_lst).to(device),
            torch.stack(targets_lst).to(device))
```

indices[1:]

is the whole masking rule. **nonzero** gives every padding position; slicing from one spares the first.

len(item) + 1

The extra slot is the end-of-text token appended before padding.

device inside collate

Moves tensors in the worker rather than the training loop — and is why this needs partial().

THINK

Should the Instruction Be Masked Too?

WHAT THE BOOK DOES

Masks padding only. The model is trained to predict the instruction text as well as the response — every token after the first is supervised.

Argument in favour: more supervised positions per example, which on 935 examples is not nothing. The instruction distribution is also worth modelling if you expect users to phrase things like your training set.

WHAT MOST LIBRARIES DO

Mask the prompt as well, so loss is computed only over the response tokens.

Argument in favour: you are not trying to teach the model to generate instructions, and with a long template the instruction can be most of the tokens — so most of your gradient is spent on text the user will always supply.

The book's supplementary material includes both. Which one wins is an empirical question, and it depends on how much of each example is template. **Run it.**

CHAPTER 7.4 · LIVE CODE

Wiring It Into the Loader

```
from functools import partial

customized_collate_fn = partial(
    custom_collate_fn,
    device=device,
    allowed_max_length=1024
)

train_loader = DataLoader(
    train_dataset,
    batch_size=8,
    collate_fn=customized_collate_fn,
    shuffle=True,
    drop_last=True,
    num_workers=0
)
```

Why partial

DataLoader calls **collate_fn(batch)** with one argument; partial freezes the rest.

1024

GPT-2's context length. Anything longer has no positional embedding.

num_workers=0

Because collate touches the GPU. Forked workers each initialising CUDA is a confusing, common failure.

CHAPTER 7.5

Why 355 Million, Not 124

Model	emb_dim	n_layers	n_heads	Parameters	Used in
GPT-2 small	768	12	12	124 M	Chapters 4–6
GPT-2 medium	1024	24	16	355 M	Chapter 7
GPT-2 large	1280	36	20	774 M	—
GPT-2 XL	1600	48	25	1558 M	—

The book's reason

"Too limited in capacity to achieve satisfactory results via instruction fine-tuning." The 124 M model gives grammatical but wrong answers.

The general principle

Instruction tuning surfaces knowledge; it does not create it. If the base model does not know the answer, no amount of formatting will make it produce one.

CHAPTER 7.5

The Base Model, Given an Instruction

PROMPT

```
### Instruction:
Convert the active sentence to passive:
'The chef cooks the meal every day.'

### Response:
```

GPT2-MEDIUM, BEFORE FINE-TUNING

```
The chef cooks the meal every day.

### Instruction:
Convert the active sentence to passive:
'The chef cooks the meal every day.'
```

It produces a Response section, then repeats the input and starts writing a new instruction. It has learned the *shape* of the template from pretraining data and nothing about the task.

CHAPTER 7.6

Two Epochs

```
optimizer = torch.optim.AdamW(model.parameters(),
                              lr=0.00005, weight_decay=0.1)

train_model_simple(
    model, train_loader, val_loader, optimizer, device,
    num_epochs=2, eval_freq=5, eval_iter=5,
    start_context=format_input(val_data[0]),
    tokenizer=tokenizer)
```

gpt2-medium, 355 M	Runtime
CPU — M3 MacBook Air	15.78 min
GPU — NVIDIA L4	1.83 min
GPU — NVIDIA A100	0.86 min

Train loss

2.637 → **0.300**

Validation loss

2.626 → **0.657**

Two epochs, no more

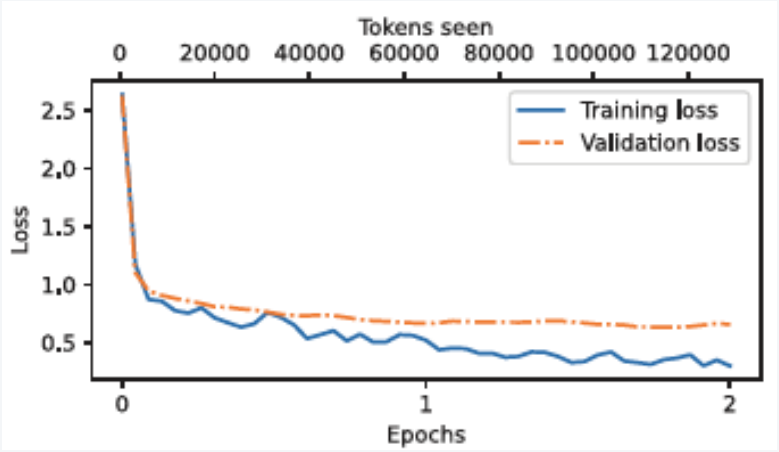
Watch what validation loss does after the first — next slide.

CHAPTER 7.6

Reading the Loss Curve Honestly

Training loss falls through both epochs. Validation loss drops sharply in the first half of epoch one, then **flattens and drifts upward**.

```
Ep 1 (Step 000000): Train 2.637 Val 2.626
Ep 1 (Step 000115): Train 0.520 Val 0.665
Ep 2 (Step 000135): Train 0.408 Val 0.677
Ep 2 (Step 000230): Train 0.300 Val 0.657
```



935 examples
is small for 355 M trainable parameters

Why stop at two
The gap between the curves *is* the overfitting. A third epoch lowers training loss and nothing else.

And yet
Instruction tuning often keeps helping after validation loss stalls: the loss measures token agreement, and quality is not token agreement.

Extracting the Responses

```
for entry in test_data:
    input_text = format_input(entry)
    token_ids = generate(
        model=model,
        idx=text_to_token_ids(input_text, tokenizer).to(device),
        max_new_tokens=256,
        context_size=BASE_CONFIG["context_length"],
        eos_id=50256
    )
    generated = token_ids_to_text(token_ids, tokenizer)
    entry["model_response"] = generated[
        len(input_text):
    ].replace("### Response:", "").strip()
```

eos_id=50256

Stop when the model emits end-of-text — the payoff for keeping one unmasked 50256 per target.

Slice off the prompt

generate() returns prompt plus continuation; the judge must score only the continuation.

Write it to disk

110 generations. You will score them more than once.

CHAPTER 7.8

Three Ways to Score a Free-Text Model

OBJECTIVE

Short-answer benchmarks

MMLU. Multiple choice, so scoring is exact. Measures stored knowledge, and is close to silent about whether the model follows instructions or writes usefully.

GOLD STANDARD

Human preference

Pairwise comparisons, aggregated into a rating — the LM Arena approach. Closest to what "good" means, and far too slow and expensive to run inside a training loop.

WHAT WE USE

LLM as a judge

A larger instruction-tuned model scores each response 0–100 against the reference. Automatable, reproducible, and inherits every bias the judge has.

Known judge failure modes: **position bias** (the first response shown wins more often), **length bias** (longer answers score higher), and **self-preference** (a judge favours text from its own family). Mitigate by randomising order, controlling for length, and never judging a model with itself.

A Local Llama 3 8B Judge

```
# POST to http://localhost:11434/api/chat with
# "options": {"seed": 123, "temperature": 0,
# "num_ctx": 2048}

def generate_model_scores(json_data, json_key, model="llama3"):
    scores = []
    for entry in tqdm(json_data, desc="Scoring entries"):
        prompt = (
            f"Given the input `{format_input(entry)}` "
            f"and correct output `{entry['output']}`, "
            f"score the model response `{entry[json_key]}`"
            f" on a scale from 0 to 100, where 100 is best."
            f" Respond with the integer number only."
        )
        scores.append(int(query_model(prompt, model)))
    return scores
```

temperature=0, seed=123

A judge must be deterministic, or the evaluation is noisier than the effect.

Runs on your laptop

Ollama wraps llama.cpp: 8B quantised, no API key.

Integer only

int(score) with no error handling. Real harnesses retry, because models do not always obey.

CHAPTER 7.9

What Comes After Instruction Tuning

DONE

1 · Pretraining

Next-token prediction on a very large corpus. Everything the model knows arrives here. Sessions 1–3.

DONE

2 · Supervised fine-tuning

Instruction–response pairs. Teaches the model the shape of an answer. This block.

PART II OF THE COURSE

3 · Preference tuning

Pairs of responses ranked by a human or a reward model. DPO, or RLHF with PPO. Teaches the model *which* answer people prefer.

Supervised fine-tuning can only imitate the answers in its dataset. Preference tuning can exceed them, because the signal is a comparison rather than a target — which is also why it is the bridge from this chapter into reinforcement learning and reasoning models. The book's DPO notebook is in the supplementary repository.

RECAP

Block II — Instruction Fine-Tuning

01 No architectural change and no new objective. The entire chapter is data engineering wrapped around the chapter 5 training loop.

02 The prompt template is a contract: train and infer with the same markers, or the model keeps writing instructions instead of answering them.

03 Pad per batch, not per dataset; shift the targets by one; mask all padding but the first end-of-text token, which is what teaches the model to stop.

04 355 M is the smallest base model that gives usable results — instruction tuning surfaces knowledge rather than creating it.

05 50.32 against 58.51 for an untuned Llama 3 8B. The base model dominates; post-training on 935 examples buys format, not capability.

PART III · APPENDIX E AND BEYOND

Beyond Full Fine-Tuning

Low-rank adapters, the variants that replaced the original recipe, and the sparse-expert models they are now applied to.

FRAMEWORK

Two Kinds of Sparsity

Full fine-tuning pays twice over. Each of this block's two halves attacks one of the payments.

	Low-rank adaptation	**Mixture of experts**
What it makes sparse	The update — trainable parameters	The forward pass — active parameters
What shrinks	Optimiser state, gradient memory, per-task storage	FLOPs and latency per token
What does not shrink	Inference cost — all weights still run	Memory — every expert stays resident
Where it is decided	At fine-tuning time	At pretraining time, in the architecture
Who pays the complexity	You, choosing rank, alpha and placement	The serving system, moving tokens between devices

THE PROBLEM

What Full Fine-Tuning Actually Costs

MEMORY, PER PARAMETER, ADAMW IN FP32

Copy	Bytes / param	355 M model
Weights	4	1.42 GB
Gradients	4	1.42 GB
Adam first moment m	4	1.42 GB
Adam second moment v	4	1.42 GB
Total, before activations	16	5.68 GB

The same arithmetic at 70 billion parameters gives roughly 1.1 TB — which is why nobody full fine-tunes a frontier model on their own hardware.

And then storage

Chapter 6 trained 7 M parameters and shipped a **475 MB** checkpoint — all 124 M weights, at 4 bytes each.

Ten customers, ten filters

Nearly five gigabytes of near-identical weights, and a cold load on every task switch.

The question LoRA asks

If the weights only move a little, why store and optimise a full-size description of the movement?

APPENDIX E.1

The Low-Rank Hypothesis

The pretrained weight matrix **W** is full rank and has to be. The *update* **ΔW** learned during fine-tuning need not be — and empirically it is not.

THE OBSERVATION

Aghajanyan et al., 2020

Measured the *intrinsic dimension* of fine-tuning: the smallest random subspace you can optimise in and still hit a target accuracy. Often a few hundred dimensions.

THE METHOD

Hu et al., 2021 — LoRA

Turned the observation into a method: constrain ΔW to rank r by construction, and optimise the two small factors instead of the large matrix.

A COMMON CONFUSION

What this is *not*

It is not a claim that the model is low rank, and not a compression of **W**. The base weights are untouched and remain full rank throughout.

MATHEMATICS

The Decomposition

```
Regular fine-tuning
      W' = W + dW dW is d_in x d_out

LoRA
      W' = W + (alpha / r) * A B

      A is d_in x r r << min(d_in, d_out)
      B is r x d_out so A B is d_in x d_out

By distributivity, applied to an input x

      x W' = x W + (alpha / r) * (x A) B

so the adapter never has to be materialised as
a full matrix.
```

One 768 × 768 projection

	Parameters
Full ΔW	589,824
LoRA, r = 16	24,576
Ratio	4.17%

Two hyperparameters

r sets capacity. **alpha** scales the adapter's contribution; the **alpha/r** factor is what makes the useful learning rate roughly independent of the rank you pick.

APPENDIX E.1

Why Keep the Factors Separate

STORAGE

One base model on disk

The pretrained weights never change. A fine-tuned task is a pair of small matrices, not a new copy of the model.

DISTRIBUTION

Adapters are megabytes

2.7 M parameters instead of 124 M — roughly **11 MB** against 475 MB for the same task in chapter 6.

DEPLOYMENT

Merge, or do not

$\mathbf{W' = W + (\alpha/r)AB}$ before deployment gives you back a plain Linear layer with zero inference overhead. Leave it unmerged and you can swap adapters per request.

That last choice reappears at the end of this half as a serving problem: merging is free at inference but forces one model copy per customer; not merging lets a single GPU batch requests for a thousand different customers, at the cost of an extra pair of matrix multiplies per layer.

APPENDIX E.4 · LIVE CODE

The LoRA Layer

```
import math

class LoRALayer(torch.nn.Module):
    def __init__(self, in_dim, out_dim, rank, alpha):
        super().__init__()
        self.A = torch.nn.Parameter(torch.empty(in_dim, rank))
        torch.nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))
        self.B = torch.nn.Parameter(torch.zeros(rank, out_dim))
        self.alpha = alpha
        self.rank = rank

    def forward(self, x):
        x = (self.alpha / self.rank) * (x @ self.A @ self.B)
        return x
```

A: Kaiming uniform

The initialisation PyTorch uses for **nn.Linear**.

B: zeros

So **AB = 0** at step zero and the adapter contributes nothing. Two slides on, why that is necessary and not merely tidy.

Eleven lines

No hidden machinery. Every LoRA implementation you will use is this, plus bookkeeping.

APPENDIX E.4 · LIVE CODE

Wrapping a Linear Layer

```
class LinearWithLoRA(torch.nn.Module):
    def __init__(self, linear, rank, alpha):
        super().__init__(); self.linear = linear
        self.lora = LoRALayer(linear.in_features,
                              linear.out_features, rank, alpha)

    def forward(self, x):
        return self.linear(x) + self.lora(x)
```

```
def replace_linear_with_lora(model, rank, alpha):
    for name, module in model.named_children():
        if isinstance(module, torch.nn.Linear):
            setattr(model, name,
                    LinearWithLoRA(module, rank, alpha))
        else: # descend
            replace_linear_with_lora(module, rank, alpha)
```

The layer is kept, not copied

self.linear is the same object, already frozen. Only the two new matrices carry gradients.

named_children, recursively

Swaps every **nn.Linear** in place — in attention, in the feed forward, and the head.

What is *not* wrapped

Embeddings and LayerNorms: not Linear layers, so the recursion skips them.

APPENDIX E.4

Freeze, Then Swap

124,441,346
trainable parameters in the loaded model

→

0
after freezing every parameter

→

2,666,528
after **replace_linear_with_lora(model, rank=16, alpha=16)**

46.7× fewer

trainable parameters than the full model, and **2.7× fewer** than chapter 6's last-block-plus-head configuration.

Rank and alpha both 16

A good default. Alpha is conventionally set to half, equal to, or double the rank; raising the rank raises capacity and the parameter count proportionally.

Adapters everywhere

W_query, W_key, W_value, out_proj, both feed-forward layers, and out_head — in all twelve blocks.

APPENDIX E.4

Why B Starts at Zero — and A Does Not

At initialisation

AB = 0, so **LinearWithLoRA(x) = linear(x)** exactly. The adapted model *is* the pretrained model.

46.25%
training

45.00%
validation

48.75%
test

Identical, digit for digit, to the pre-fine-tuning numbers in chapter 6. That equality is the check that your adapter is wired in correctly — run it before you train anything.

Why not initialise both to zero?

The gradient of the loss with respect to **A** contains a factor of **B**, and the gradient with respect to **B** contains a factor of **xA**. If both are zero, both gradients are zero, and they stay zero forever.

Exactly one must be zero

so the product vanishes at the start, and exactly one must not, so the product can move. Zero **B**, random **A** is the conventional choice.

APPENDIX E.4

LoRA Against Full Fine-Tuning

	Chapter 6 — last block + norm + head	Appendix E — LoRA on every Linear
Trainable parameters	7,090,946	2,666,528
Learning rate	5e−5	8e−4 — sixteen times larger
Runtime, M3 MacBook Air	5.65 min	12.10 min
Training accuracy	97.21%	100.00%
Validation accuracy	97.32%	96.64%
Test accuracy	95.67%	98.00%

LoRA was slower here, and that is not a typo. At 124 M the adapters cost more on the forward pass than they save on the backward one. The saving is in optimiser memory and checkpoint size; it becomes a speed saving only once gradients dominate.

EVIDENCE · THINKING MACHINES LAB, 2025

“LoRA Without Regret”

Yes — but only inside a regime the study defines precisely.

1 · LOW-REGRET

It matches full fine-tuning

Adapters on every layer, and a dataset the adapter has the capacity to absorb. Same loss curve, same sample efficiency, same final performance — at about two-thirds of the FLOPs per pass.

2 · CAPACITY-BOUND

The dataset outgrows the adapter

More information in the data than the adapter can store. Raising the rank helps; nothing else does. Most “LoRA is worse” results live here.

3 · BATCH-BOUND

Large batches train less efficiently

The gap grows with batch size and does *not* close as rank rises. It comes from optimising a product of two matrices rather than one.

WHAT THEY RAN

LR sweeps over 14 Llama and Qwen models on Tulu3 · SFT on OpenThoughts3 · RL on MATH, GSM and DeepMath-103K · a dense base (Llama-3.1-8B) and a sparse one (Qwen3-30B-A3B).

EVIDENCE · CONDITION 1

Put the Adapters Where the Parameters Are

LLAMA-3.1-8B, MATCHED PARAMETER BUDGET

Adapters on	Rank	Trainable	Result
MLP only	128	0.24 B	better
Attention only	256	0.25 B	worse

Twice the rank and a slightly larger budget, and attention-only still loses. Adding attention adapters *on top of* MLP adapters buys nothing further.

Why

LoRA reproduces full fine-tuning's gradient geometry only where the adapters cover the layers that dominate it — the layers holding the parameters. In a transformer block that is the MLP: two thirds of the weights.

Replicated

On Qwen3-30B-A3B (a sparse MoE base), on OpenThoughts3 at rank 256, and on RL over MATH.

The rule: adapt every linear layer. The original LoRA paper adapted only the attention projections, and much fine-tuning code still does. Appendix E's **replace_linear_with_lora** already gets this right.

EVIDENCE · CONDITION 2

How Much Can an Adapter Hold?

capacity ≈ 1–2 bits × trainable parameters	what the weights can store
demand ≈ 1 bit × tokens	supervised — 0.69 nats per token
demand ≈ 1 bit × episodes	reinforcement learning — one scalar reward each

Count both sides in bits. LoRA matches full fine-tuning while **capacity > demand**.

Supervised learning is hungry

Every token of every answer is a target: O(tokens) bits. A large SFT corpus can outgrow a small adapter.

Reinforcement learning is not

O(1) bit per episode. Rank 1 matched full fine-tuning on MATH, GSM and DeepMath-103K, at the same peak and over a *wider* band of learning rates.

Worked example — RL on MATH

10,000 problems × 32 samples = 320,000 episodes, so ≈ **320,000 bits**. A rank-1 LoRA on the 8 B model holds ≈ **3 M parameters** — ten times that, at a bit each.

EVIDENCE · IN PRACTICE

The Numbers You Actually Set

OPTIMISATION

Learning rate ≈ 10× full fine-tuning

Fitted across 14 Llama and Qwen models on Tulu3: a multiplier of **9.8**, with the same dependence on hidden size as full fine-tuning. Nearer 15× under a hundred steps, and roughly rank-independent. Appendix E's 8e−4 against chapter 6's 5e−5 is 16×.

COMPUTE

About two-thirds of the FLOPs

Per pass over an $N \times N$ matrix at rank R: full fine-tuning costs $3N^2$ multiply-adds, LoRA costs $2N^2 + 6NR$. Plot performance against compute rather than against steps and LoRA is clearly ahead.

THE ONE REAL GAP

Small batches, for both

LoRA pays a larger penalty than full fine-tuning as batch size grows, at any rank — the product **BA** optimises less kindly than a full matrix. Both reach their best loss at small batches anyway.

HYPERPARAMETERS

They could not beat the library defaults

$\alpha = 32$ · **A** initialised uniform at $1/\sqrt{d_{in}}$ · **B** at zero · the same learning rate for **A** and **B**. The HuggingFace PEFT defaults survived the whole sweep.

BEYOND THE BOOK

The PEFT Family

Method	Year	What it changes	What it buys	What it costs
LoRA · Hu et al.	2021	ΔW constrained to rank r	10–100× fewer trainable parameters; mergeable at deploy	Rank and alpha to choose
QLoRA · Dettmers et al.	2023	Frozen base weights held in 4-bit NF4	65 B fine-tuned on one 48 GB GPU	Slower steps; dequantisation on the fly
rsLoRA · Kalajdzievski	2023	Scale by $\alpha/\sqrt{r}$ instead of α/r	High ranks train stably instead of collapsing	Nothing at small rank
DoRA · Liu et al.	2024	ΔW split into magnitude and direction	Consistently above LoRA, with no inference overhead	An extra normalisation each step
LoRA+ · Hayou et al.	2024	A larger learning rate for B than for A	Faster convergence at the same rank	One more ratio to tune

DORA · LIU ET AL., 2024

Magnitude and Direction

Any weight matrix factors into a scale and a set of unit-norm columns

$$W = m * V / ||V||$$

m is a per-column magnitude vector
$V / ||V||$ is the direction

Full fine-tuning moves both, and moves them differently from each other.

LoRA is a single update that entangles them.

DoRA trains m directly, and uses LoRA only for the directional part.

Where it came from

An analysis of how magnitude and direction move under full fine-tuning against LoRA. The patterns differ; DoRA's resembles full fine-tuning's.

Consistently above LoRA

On LLaMA, LLaVA and VL-BART — ICML 2024, oral.

No inference overhead

It merges back into a plain weight matrix.

SERVING

One Base Model, Thousands of Adapters

MERGED

```
h = x @ W_merged
```

Zero inference overhead. But the merged weights are task-specific, so every customer needs their own resident copy of the whole model, and a batch cannot mix customers.

UNMERGED

```
h = x @ W + (alpha/r) * (x @ A) @ B
```

One shared base model; each request brings its own tiny **A** and **B**. Two thin matmuls per layer, and a much harder kernel.

S-LORA · 2023

Unified paging

Adapters live in host memory and page in. Weights of differing ranks and KV-cache tensors of differing lengths share *one* pool.

KERNELS

Heterogeneous batching

Custom CUDA kernels apply a different adapter to each sequence in one batch.

RESULT

Up to 4× throughput

over HuggingFace PEFT and vLLM, serving thousands of adapters on one GPU.

Where the Parameters Actually Are

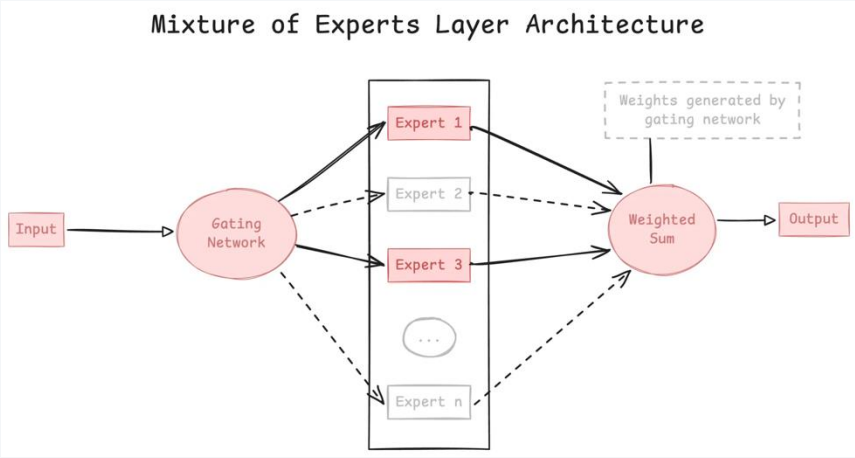
Counting one GPT-2 transformer block, ignoring biases and norms.

Component	Parameters per block	Share
Attention — 4 × (768 × 768)	2,359,296	33%
Feed forward — 2 × (768 × 3072)	4,718,592	67%

Exactly two to one, by construction: the feed-forward block expands to **4×** the embedding width and back. This ratio holds across essentially every transformer you will meet.

So sparsify the feed forward

A mixture-of-experts layer replaces that single dense block with *N* copies of experts, and runs *k* of them per token.



The Layer

```
Dense feed forward

    h = FFN(x) always, for every token


Mixture of experts

    s = router(x) affinity, one per expert
    T = TopK(s, k) k winners
    g_i = normalise(s_i) for i in T, else 0

    h = sum over i in T of g_i * Expert_i(x)


    N experts held in memory
    k experts executed k << N
```

The decoupling

Total parameters scale with **N**; FLOPs per token with **k**. Capacity and cost stop being one number.

Sparsity ratio N / k

Mixtral: **4**. DeepSeek-V3: **32**. Kimi K2: **48**. Three years, straight up.

Shared experts

Some designs keep experts always on, so common knowledge is stored once rather than duplicated.

MATHEMATICS

The Router

$$s_{i,t} = \sigma(x_t \cdot e_i)$$ affinity of token t for expert i

$$T_t = \mathrm{TopK}(s_{i,t}, k)$$ the k winners

$$g_{i,t} = s_{i,t} / \Sigma_{j \in T_t} s_{j,t}$$ gate, for $i \in T_t$; zero otherwise

$$h_t = x_t + \Sigma_i g_{i,t} \cdot E_i(x_t)$$ what the layer returns

x_t	hidden state of token *t*, width *d*
e_i	row *i* of the router matrix W_r ($d \times N$): expert *i*'s centroid
σ	per-expert sigmoid (DeepSeek-V3), or softmax over all *N* (GShard, Mixtral)
N , k	experts in memory, and experts run per token: $k \ll N$. E_i is a full feed-forward block.

Why sigmoid when N is large

A softmax over 256 experts drives every gate toward zero as *N* grows; a per-expert sigmoid scores each independently.

The router is one matrix

$d \times N$. At width 7,168 with 256 experts, under two million parameters decide how tens of billions get used.

TopK is not differentiable

Gradients reach the router only through the *chosen* experts. One never chosen never improves.

MIXTURE OF EXPERTS

Why Routing Collapses

An expert that is slightly better early takes more tokens, more gradient, improves faster, and takes more tokens still. The rest become dead weight, and the devices holding them idle at every collective.

THE CLASSICAL FIX — AN AUXILIARY LOSS

$$L = \alpha \cdot N \cdot \Sigma_i f_i P_i$$ minimised at $L = \alpha$, when every expert is equally loaded

$$f_i = (1/T)\, \Sigma_t\, 1[\, i \in T_t \,]$$ the load

$$P_i = (1/T)\, \Sigma_t\, s_{i,t}$$ the probability

- f_i — fraction of the batch's T tokens dispatched to expert i — an indicator count, so **no gradient**.
- P_i — mean router probability assigned to expert i over those same tokens — differentiable, but blind to actual load.
- α , N — balance weight, and the expert count that holds the loss at one size as N changes. Switch Transformer: $\alpha = 10^{-2}$.

Why a count times a probability?

Neither works alone: $\mathbf{f}_i$ knows which experts are overloaded but has no gradient; $\mathbf{P}_i$ has the gradient but not the load. The product scales each expert's gradient by how busy it is.

And its problem

A gradient on the LM objective that has nothing to do with language modelling. Too small and the router collapses; too large and you buy balance with quality.

Auxiliary-Loss-Free Balancing

```
Add a per-expert bias to the SELECTION score only

    s'(i,t) = s(i,t) + b_i
    T = TopK( s' , k )
but combine with the UNBIASED gate

    h = sum over i in T of g( s(i,t) ) * Expert_i(x)

and update the bias from observed load, after each step

    b_i <- b_i + eta * sign( n_bar - n_i )

    n_i tokens that went to expert i
    n_bar the average over experts
```

No gradient at all

b moves by a rule, not by backpropagation. It never enters the loss, so it cannot trade quality for balance.

Bias the choice, not the weight

The bias decides *which* experts run; the unbiased affinity decides *how much* each contributes.

Just a controller

sign of the load error times a step size — a proportional controller on expert utilisation.

MIXTURE OF EXPERTS

Expert Topology, Three Generations

Shape	Model	Year	Total	Active	Routed experts	Active / token
Coarse-grained	Mixtral 8×7B	2023	46.7 B	12.9 B	8	2
Fine-grained + shared	DeepSeek-V2	2024	236 B	21 B	—	—
Fine-grained + shared	DeepSeek-V3	2024	671 B	37 B	256 + 1 shared	8
Ultra-sparse	Kimi K2	2025	1.04 T	32 B	384	8

Read the total and active columns together. Kimi K2 holds a trillion parameters and runs thirty-two billion of them per token — roughly what a dense 32 B model costs to serve, from a model that has seen far more capacity during training.

Two Ideas That Made Fine-Grained Routing Work

IDEA 1

Fine-grained segmentation

Split each expert into several narrower ones and activate proportionally more of them. The FLOPs per token are unchanged; the number of ways to *combine* experts is not.

IDEA 2

Shared expert isolation

Keep one or more experts always on. Knowledge that every token needs gets stored once, instead of being duplicated inside each routed expert that might be selected.

28

ways to choose 2 experts from 8 — Mixtral's routing space

→

4 × 10^14

ways to choose 8 from 256 — DeepSeek-V3's. Same arithmetic per token, four hundred trillion times more ways to compose it.

And it pays

Kimi K2 report that reaching a given validation loss at sparsity 48 takes **1.69×** fewer FLOPs than at sparsity 8.

SYNTHESIS

Where the Two Halves Meet

ADAPTING AN MOE MODEL

The current advice is to put adapters wherever the parameters are. On a fine-grained MoE that means *inside the experts* — and each expert sees only the fraction of tokens routed to it, so an adapter on expert i receives roughly $\mathbf{k/N}$ of the gradient traffic an equivalent dense adapter would get.

An open question for your mini-project

Does an MoE model need a higher rank, a higher learning rate, or more data to adapt as well as a dense model of the same active size? The capacity argument suggests a testable prediction.

ROUTING OVER ADAPTERS

Run the construction the other way. A bank of LoRA adapters with a router in front of them *is* a mixture of experts — a very cheap one, because each expert is two thin matrices rather than a full feed-forward block.

Why anyone would

Multi-task serving without choosing an adapter per request; and a hedge against the single-adapter capacity ceiling. An active research line rather than settled practice.

RECAP

Block III — Beyond Full Fine-Tuning

01 LoRA constrains the update, not the model: **$\Delta W = (\alpha/r)AB$**, with **B** at zero so the adapted model starts identical to the base model.

02 2.7 M trainable parameters beat chapter 6's 7.1 M on test accuracy, at a sixteen times larger learning rate — and were slower, because at 124 M the adapters cost more than they save.

03 Adapters belong on every layer, especially the MLP; the useful learning rate is about ten times full fine-tuning's; and LoRA matches full fine-tuning until the dataset carries more information than the adapter can hold.

04 Mixture of experts makes the *forward pass* sparse rather than the update. Capacity scales with N, cost with k, and balance is enforced by a bias controller outside the loss.

05 Neither technique is free where the other one pays: LoRA does nothing for inference cost, and MoE does nothing for memory.

READING

Before Session 5

Required	Raschka, chapters 6 and 7	
Required	Raschka, appendix E	
Recommended	Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models*	arXiv 2106.09685. Sections 4 and 7 in particular.
Recommended	Thinking Machines Lab, *LoRA Without Regret*	The capacity argument and the 10× learning-rate rule.
Optional	Dettmers et al., *QLoRA* · DeepSeek-V3 technical report, § MoE	arXiv 2305.14314 and arXiv 2412.19437. Read the DeepSeek report for the loss-free balancing rule.

SESSION 4 · CLOSE

Stage 3 is finished. You have a classifier, an assistant, and a way to make both cheap.

That closes Part I. Everything so far has trained a model to imitate text that already exists — the corpus in pretraining, the reference answers in fine-tuning. Session 5 opens Part II, where the supervision signal is a comparison rather than a target, and the model can become better than anything it was shown.

NEXT · SESSION 5 · REASONING MODELS